

DEAR ANGEL NAIL STUDIO

ENTREGA TÉCNICA

Manual técnico y de operación

Instalación Docker, seguridad, respaldos, restauración, pruebas y mantenimiento.

01 · ARQUITECTURA

Componentes y responsabilidades

SERVICIO	RESPONSABILIDAD
web	Next.js, interfaz responsive, proxy de mismo origen y PWA.
api	NestJS, permisos, reglas de negocio, migraciones y auditoría.
worker	Consumer y programadores BullMQ. Ejecuta casos de uso mediante endpoints internos de la API autenticados con un secreto compartido.
postgres	Fuente de verdad; restricciones SQL evitan traslapes.
redis	Colas y coordinación asíncrona.
minio	Catálogo, cotizaciones, marca y comprobantes privados.
backup	Copia semanal consistente de PostgreSQL y MinIO con checksum.
stable-preview	Túnel HTTPS fijo de ngrok para demostración; existe únicamente en el overlay público.

La solución es un monolito modular en TypeScript. Las reglas críticas viven en API y base de datos; el frontend no es el único control.

Datos persistentes

Los volúmenes `postgres_data`, `redis_data` y `minio_data` sobreviven a recreaciones. La carpeta `backups/` contiene archivos portables; está ignorada por Git.

No ejecutes `docker compose down -v` durante la operación: elimina la base y los archivos persistentes del proyecto seleccionado.

Tareas asíncronas

Redis conserva la cola. El worker programa entregas cada 30 segundos, recordatorios cada 5 minutos, vencimientos cada minuto y retención de comprobantes cada 6 horas. Con `BACKGROUND_JOBS_MODE=worker`, la API no inicia programadores duplicados. API y worker deben compartir exactamente el mismo `WORKER_SHARED_SECRET`.

02 · PUESTA EN MARCHA

Instalación reproducible

Requisitos

- Windows 10/11 con WSL 2.
- Docker Desktop en contenedores Linux.
- Node.js 22 o posterior y npm 10.9 o posterior para scripts y desarrollo en host.
- 8 GB de RAM recomendados.
- Puertos 3000, 3001, 3002, 5432, 6379, 9000 y 9001 disponibles.

Inicio local

```
Copy-Item .env.example .env
docker compose up -d --build
docker compose ps
Invoke-WebRequest http://localhost:3000
```

La API aplica migraciones automáticamente antes de iniciar. Todos los servicios deben mostrar `healthy` antes de operar. Los puertos del Compose local se enlazan sólo a `127.0.0.1`.

Enlace fijo

Copia `.env.public.example` como `.env.public`. Configura allí `NGROK_AUTHTOKEN`, `NGROK_DOMAIN`, las URLs HTTPS y todos los secretos que exige el overlay público. El archivo resultante está ignorado por Git. Después:

```
Copy-Item .env.public.example .env.public
docker compose down
docker compose --env-file .env.public -f compose.yaml -f docker/compose.public.yaml --profile stable-preview up -d --build worker backup stable-preview
docker compose --env-file .env.public -f compose.yaml -f docker/compose.public.yaml --profile stable-preview logs stable-preview
```

El perfil temporal `preview` usa el mismo patrón con `--profile preview`. No se debe ejecutar ninguno de los dos perfiles sin `--env-file .env.public` y `docker/compose.public.yaml`.

El overlay usa el proyecto `dear-angel-public` y volúmenes separados, de modo que PostgreSQL nace con las credenciales públicas. Local y público comparten puertos y no pueden estar activos simultáneamente. `docker compose down` detiene el local sin borrar sus volúmenes; nunca agregues `--volumes` para cambiar de entorno.

Datos de demostración

```
$env:DEMO_DATA_ENABLED='true'  
npm run demo:seed  
Remove-Item Env:DEMO_DATA_ENABLED
```

El seed es explícito e idempotente. Crea dos manicuristas, tres clientes, 15 citas, tres anticipos, visitas y un cupón. No se activa solo.

03 · CONFIGURACIÓN SEGURA

Entorno e integraciones

GRUPO	VARIABLES CLAVE
Base	POSTGRES_PASSWORD, DATABASE_URL, MINIO_ROOT_PASSWORD, OTP_PEPPER
Aplicación	PUBLIC_APP_URL, CORS_ORIGIN, SESSION_TTL_DAYS, WORKER_SHARED_SECRET
WhatsApp	PHONE_NUMBER_ID, ACCESS_TOKEN y plantillas aprobadas; OTP y recuperación usan AUTHENTICATION con botón Copiar código
Correo	SMTP_HOST, SMTP_USER, SMTP_APP_PASSWORD
Google	CLIENT_ID, CLIENT_SECRET, REDIRECT_URI, INTEGRATION_ENCRYPTION_KEY

En producción la API rechaza secretos locales, CORS comodín, URLs públicas sin HTTPS y credenciales incompletas de una integración habilitada. También rechaza el OTP de depuración. Un código mock está apagado por defecto y sólo puede mostrarse temporalmente en desarrollo con `OTP MOCK DEBUG ENABLED=true`. Actívalo únicamente para probar registro o recuperación, recrea la API y vuelve a `false` al terminar.

La interfaz muestra siempre país y lada y envía teléfonos E.164; la API no presupone México si una llamada directa omite la lada. En modo real, Meta envía desde el número empresarial asociado a `WHATSAPP_PHONE_NUMBER_ID` hacia el número de prueba. El estado *Aceptado por proveedor* confirma una respuesta satisfactoria de Meta o SMTP, no la entrega o lectura final; esa confirmación requiere webhooks del proveedor.

Docker frente a ejecución en host

`.env.example` usa `localhost` para PostgreSQL, Redis y MinIO; `compose.yaml` inyecta sus hosts internos directamente. Al ejecutar API y worker con Node.js en Windows, exporta `DATABASE_URL`, `REDIS_URL`, las variables de MinIO, `API_INTERNAL_URL` y `WORKER_SHARED_SECRET` en la terminal, porque el worker no carga automáticamente el `.env` raíz. Para publicación usa la plantilla separada `.env.public.example`; nunca reutilices los secretos locales.

Controles implementados

- Contraseñas scrypt con sal aleatoria.
- Sesiones HttpOnly, SameSite=Lax y Secure en producción.
- Roles en backend y rutas administrativas exclusivas.
- Límites de intentos para registro, login y recuperación.
- Comprobantes sin URL pública; autorización por cada descarga.
- Tipos y tamaños limitados; SVG no permitido para cargas administrables.
- CSP, frame-ancestors, nosniff, HSTS, Referrer y Permissions Policy.
- Exportaciones neutralizadas contra fórmulas de hoja de cálculo.
- Tokens de Google cifrados con AES-256-GCM.

04 · RESPALDO

Copia, verificación y restauración

El contenedor `backup` genera una copia al iniciar y después cada 604800 segundos. Conserva 90 días por defecto y, si un intento falla, reintenta después de 300 segundos. El `.tar.gz` incluye dump, objetos, manifiesto y hashes internos; el checksum externo se genera como archivo adyacente. La verificación exige ambos. El marcador de salud se conserva junto a los respaldos y lo actualizan tanto el daemon como una copia manual exitosa.

Crear y verificar

```
npm run backup:now
powershell -ExecutionPolicy Bypass -File scripts/verify-backup.ps1 `
  -BackupFile .\backups\dear-angel-FECHA-SUFIJO.tar.gz
npm run backup:test-restore
```

`backup:test-restore` crea una base y bucket temporales con nombres validados, restaura todo, compara los objetos con el manifiesto, informa los conteos de usuarios y migraciones aplicadas, y elimina únicamente esos destinos temporales.

Respaldo del entorno público

Los scripts PowerShell anteriores apuntan al proyecto local y se usan con el público detenido. En `dear-angel-public`, deja activo el daemon `backup`. Para crear y verificar de inmediato una copia pública usa:

```
docker compose --env-file .env.public -f compose.yaml -f docker/compose.public.yaml run --rm --no-deps
backup once
docker compose --env-file .env.public -f compose.yaml -f docker/compose.public.yaml --profile tools run
--rm -e "BACKUP_FILE=/backups/dear-angel-FECHA-SUFIJO.tar.gz" restore verify
```

No uses `restore-backup.ps1` contra el proyecto público. Una restauración pública requiere revisar explícitamente el proyecto y el archivo objetivo.

Restaurar operación local

```
powershell -ExecutionPolicy Bypass -File scripts/restore-backup.ps1 `
  -BackupFile .\backups\dear-angel-FECHA-SUFIJO.tar.gz `
  -ConfirmRestore
```

Destructivo: la restauración operativa reemplaza la base y el bucket actuales. Primero conserva otra copia, verifica el archivo y confirma el objetivo exacto.

Retención de comprobantes

La API revisa cada seis horas los comprobantes con un año cumplido. El objeto se elimina de MinIO y la base conserva la evidencia de purga, sin borrar la cita o la decisión del anticipo.

05 · CALIDAD

Pruebas y liberación

Verificación diaria

```
npm run quality
npm audit --omit=dev
```

Los hooks de calidad regeneran Prisma Client antes de typecheck, lint, pruebas y build. No hace falta ejecutar `db:generate` por separado para esas tareas.

Prueba integrada

```
npm run test:e2e
```

Comprueba portada, PWA, cabeceras, salud, sesiones de cliente/manicurista, permisos, privacidad de comprobantes, cita manual a cualquier minuto, rechazo de traslape, dashboard, cuatro reportes, CSV, XLSX y auditoría. Los registros temporales se limpian al terminar.

Estado de la candidata: el 14 de agosto de 2026 pasaron calidad integral, reconstrucción y salud Docker, E2E, respaldo/restauración aislada y clean-install. La entrega externa real se confirma después de incorporar las credenciales propias de Meta, SMTP y Google.

Lista antes de mostrar

- Todos los contenedores saludables.
- Migraciones aplicadas.
- Respaldo reciente verificado.
- Enlace público HTTP 200.
- Sin errores recientes en logs de API/web.
- Vista móvil sin desbordamiento.
- Credenciales demo diferentes de las reales.

Observabilidad local

```
docker compose ps
docker compose logs --tail 200 api web worker backup
docker compose exec postgres pg_isready
```

`npm run test:clean-install` no relanza los túneles públicos. Si eran necesarios, vuelve a iniciarlos de forma explícita con `--env-file .env.public` y ambos archivos Compose después de validar la instalación local.

Diagnóstico y recuperación

SÍNTOMA	ACCIÓN
Docker no inicia tras apagar Windows	Reinicia WSL/Docker; si WSL no responde, reinicia Windows. No borres volúmenes.
El enlace no abre	Confirma que laptop, Docker, web y stable-preview estén encendidos y que el perfil se haya iniciado combinando el Compose base y el overlay público.
API no está healthy	Revisa logs y salud de PostgreSQL, Redis y MinIO.
WhatsApp/correo falla	La operación interna permanece; revisa entregas y variables del proveedor.
Migración falla	No edites el historial aplicado. Conserva logs y restaura en entorno aislado antes de intervenir.
Archivo de respaldo falla hash	No lo restaures. Usa la copia anterior verificada.

Frecuencia recomendada

- Diario: health y entregas fallidas.
- Semanal: confirmar nuevo respaldo y espacio disponible.
- Mensual: ejecutar restauración aislada y actualizar dependencias con revisión.
- Antes de cada entrega: suite completa, capturas y ensayo Docker.

Documentos relacionados

- README.md
- ROADMAP.md y PROGRESO.md
- ARQUITECTURA.md y DECISIONES.md
- ESPECIFICACION_REQUERIMIENTOS_CASOS_USO.pdf
- MANUAL_USUARIO.pdf

